Informe de seguridad y calidad en el desarrollo de software

Encargo 3: pruebas de seguridad, automatización y mejora continua

Caso SmartLogix

Asignatura: ISY1102 - Seguridad y calidad en el desarrollo de software

Evaluación: Encargo 3 (tercera etapa del caso semestral)

Carrera: [completar]

Sección: 004V

Docente: Manuel Reyes Ferrada

Fecha: 14 de octubre de 2026

Integrantes

Nicolás Castillo

Benjamín Catalán

Benjamín Quiroz

Índice

1. Introducción 4
1.1 Contexto y objetivo del encargo 4
1.2 Alcance y estructura 4
1.3 Ajustes a los encargos previos 4
2. Criterios de calidad, seguridad y cumplimiento 5
3. Enfoques y herramientas de pruebas de seguridad 6
4. Diseño de los controles de seguridad automatizados 6
5. Resultados de las pruebas de seguridad 8
5.1 Análisis dinámico (OWASP ZAP) 8
5.2 Análisis estático (SAST) 9
5.3 Análisis de dependencias (SCA) 10
5.4 Batería de casos de prueba 11
6. Resultados de las pruebas automatizadas 12
6.1 Suites automatizadas y cobertura 12
6.2 Pruebas E2E y de API 12
6.3 Pipeline de integración continua 13
6.4 Eficiencia de la automatización 14
6.5 Regresión: qué prueba detecta cada defecto corregido 14
7. Incidentes identificados, criticidad y propuestas de mitigación 15
7.1 Propuestas de mitigación para los defectos abiertos 18
8. Plan de mantenimiento y preservación de la calidad, la seguridad y la normativa 19
8.1 Política de regresión y de cambios 19
8.2 Preservación de los aspectos normativos y legales 20
9. Análisis ético y normativo de los resultados 20
9.1 Análisis ético 20
9.2 Análisis normativo 21

10. Conclusiones y propuestas para preservar la calidad..... 21

Anexo A. Tarjetas de los casos de prueba (ejecución final) 21

Anexo B. Reproducibilidad 24

Anexo C. Mapa de evidencias 24

Anexo D. Responsables de la ejecución 25

Índice generado por Word (clic derecho y 'Actualizar campos').

1. Introducción

1.1 Contexto y objetivo del encargo

SmartLogix es un eCommerce de perfumes con arquitectura de microservicios: un frontend en React 18, un BFF (gateway) con JWT y tres microservicios en Spring Boot 3.2 y Java 21 (inventario, pedidos y envíos). El Encargo 3 es la última etapa del caso: valida el sistema de forma global, ejecuta pruebas de seguridad, automatiza las pruebas funcionales y no funcionales, y establece cómo preservar la calidad, la seguridad y el cumplimiento normativo en futuras versiones.

El objetivo es comprobar, con evidencia reproducible, qué mejora el trabajo de las etapas anteriores y qué queda pendiente. Para ello se compara la versión de referencia (rama main, stack del Encargo 2) con la versión final (rama v2), con el mismo ejecutor y las mismas herramientas.

1.2 Alcance y estructura

Incluye el BFF, los tres microservicios, el frontend, el despliegue en Docker y el pipeline de integración continua. Quedan fuera la infraestructura de nube, la carga concurrente a escala y las pruebas en dispositivos móviles. El informe contiene: ajustes a los encargos previos (sección 1.3), enfoques de seguridad (sección 3), controles automatizados (sección 4), resultados de seguridad (sección 5), resultados de automatización (sección 6), incidentes y propuestas de mitigación (sección 7), plan de mantenimiento (sección 8), análisis ético y normativo (sección 9) y conclusiones (sección 10).

1.3 Ajustes a los encargos previos

La revisión técnica de los encargos anteriores (Encargo 1 y Encargo 2) permitió detectar afirmaciones que el código no respaldaba. La siguiente tabla resume cada una y el ajuste aplicado. No se recibió retroalimentación escrita del docente sobre esas entregas; los ajustes se basan en la verificación contra el código y la ejecución.

Afirmación en el encargo previo	Verificación con el código o la ejecución	Ajuste en este encargo
Cobertura de 87,4 % (backend) y 84,1 % (frontend)	El gate de JaCoCo estaba inactivo y el frontend mide 68,5 % de statements.	Gate corregido (INC-07). La cobertura de cada módulo se mide en la sección 6.1, sobre la versión final.
Errores devueltos con 404 o 500 sin detalles	Todos los errores de negocio respondían 404 y enviaban el mensaje de la excepción.	Respuestas 400/404/409/429 controladas y sin mensajes internos (INC-03, INC-05, INC-20).
Sin token, HTTP 401	La respuesta real era 403 en la base.	401 con mensaje JSON (INC-04).

Contraseñas 'protegidas' (ver Encargo 2)	Se guardaban en texto plano.	BCrypt (INC-01).
Microservicios protegidos por el BFF	Eran accesibles directamente desde la red.	Validación de JWT en cada servicio (INC-15).
Búsqueda de productos disponible	El BFF no exponía los filtros de inventario.	Endpoints añadidos (INC-17).
Ley 19.628 como marco vigente	La Ley 21.719 la reforma; BCN indica vigencia el 1-dic-2026, con una propuesta de postergación a 1-dic-2027.	Se analiza la ley reformada y se marca como pendiente la confirmación de la fecha (sección 9).

2. Criterios de calidad, seguridad y cumplimiento

Los criterios son los del Encargo 2, ampliados con los requisitos de seguridad y automatización que surgieron del análisis. Cada requisito tiene un código, un origen declarado y al menos un caso de prueba. El conjunto no es un ERS oficial, y así se declara.

Código	Requisito	Origen	Caso o prueba que lo verifica
REQ-SEG-01	Autenticación JWT; rechazo de accesos sin token o con credenciales inválidas.	Encargo 2	CP-01, CP-02, API-01, API-06
REQ-SEG-02	Contraseñas almacenadas como hash.	Encargo 2 (§2.2)	CP-10; AuthServiceTest
REQ-SEG-03	Errores sin detalles internos.	Encargo 2 (§2.2)	CP-11
REQ-SEG-04	CORS restringido a los orígenes definidos.	Encargo 2 (§2.2)	CP-12
REQ-SEG-05	Secreto JWT fuera del código.	Hallazgo	CP-13
REQ-SEG-06	Dependencias del frontend sin critical o high en producción.	Análisis SCA	CP-16 (completo); npm audit --omit=dev
REQ-SEG-07	Microservicios accesibles solo con token firmado.	Hallazgo (INC-15)	CP-18, CP-20; SecurityIntegrationTest
REQ-SEG-09	Bloqueo tras intentos fallidos de login.	Hallazgo (INC-13)	CP-21; AuthServiceTest
REQ-SEG-10	Cabeceras de seguridad (CSP y aislamiento).	DAST (INC-21)	ZAP baseline; cabeceras de nginx
REQ-FUNC-02	Descuento WEB 5 %, MARKETPLACE 10 %; tipo inválido rechazado.	Encargo 2	CP-03, CP-04, API-03
REQ-FUNC-03	Búsqueda por marca, fragancia, precio, stock y nombre.	Encargo 2	CP-06
REQ-FUNC-04	Alta de producto con identificador.	Ev1 §3.2	CP-19; ProductSeedIdIntegrationTest
REQ-FUNC-05	Actualización de estado a través del BFF, sin inyección en la URL.	Hallazgo (INC-19, INC-22)	API-04, CP-22; StatusUrlEncodingTest
REQ-RES-01	Fallback del Circuit Breaker ante fallo del carrier.	Encargo 2	CP-05
REQ-VAL-01 / 02	Validación de entradas; parámetro obligatorio ausente = 400.	Encargo 2 / hallazgo (INC-20)	CP-07, CP-08, API-05, CP-23
REQ-NFR-01	Lectura p95 < 500 ms; escritura p95 < 1000 ms.	Encargo 2 (§2.1)	CP-09
REQ-CAL-01 / 02	Cobertura >= 60 % por módulo; sin prioridad 1 en SpotBugs sin tratamiento.	Encargo 2 / criterio propio	CP-14, CP-15
REQ-DEP-01	Despliegue reproducible en contenedores.	Pauta	CP-17

3. Enfoques y herramientas de pruebas de seguridad

Las pruebas de seguridad se organizan en seis enfoques complementarios. Ninguno basta por sí solo: el análisis estático no ve la configuración en ejecución, y el dinámico no ve el código que no se expone por HTTP. La tabla indica la función de cada enfoque en la detección de vulnerabilidades y su aplicación en el proyecto.

Enfoque	Función en la detección	Herramienta y ejemplo en SmartLogix
SAST (análisis estático de código)	Detecta patrones inseguros sin ejecutar el código: configuración de CSRF, inyección y registros.	SpotBugs + FindSecBugs (CSRF deshabilitado, CRLF en logs); SonarQube (S4502); ESLint con eslint-plugin-security (inyección de objetos).
DAST (análisis dinámico)	Ataca la aplicación en ejecución y observa cabeceras, errores y respuestas del servidor.	OWASP ZAP baseline sobre el frontend y escaneo de la API con token válido. Detectó errores 500 y 503 en la API (INC-19, INC-20, y el caso que llevó a INC-22) y la ausencia de CSP (INC-21).
SCA (composición de software)	Identifica vulnerabilidades conocidas en dependencias de terceros.	npm audit (producción y completo). Separa dependencias de ejecución de las de build (INC-09).
Pruebas de caja negra de seguridad	Verifican el comportamiento observable frente a entradas hostiles o no autorizadas.	Batería de casos CP-01 a CP-23 y pruebas de API contra el BFF (sin token, token falso, entradas inválidas).
Pruebas de configuración	Comprueban secretos, orígenes permitidos y cabeceras en los archivos de despliegue.	CP-12 (CORS), CP-13 (secreto), CP-17 (Compose) y la revisión de nginx.conf.
Pruebas de regresión de seguridad	Garantizan que una corrección no se pierda en versiones posteriores.	Las pruebas que fallaban en main y pasan en v2 quedan en el pipeline (sección 6.5).

Criterio de selección: se eligieron herramientas gratuitas, con imagen oficial o plugin de Maven y npm, y adecuadas al lenguaje (Java 21) y al framework (Spring Boot 3.2, React 18). Se descartó Burp Suite Community por sus limitaciones de automatización; se usa ZAP, que se ejecuta en CI sin licencia.

4. Diseño de los controles de seguridad automatizados

La figura 1 muestra el pipeline de GitHub Actions. Cada etapa tiene una puerta de calidad o de seguridad, y cada puerta se vincula con un criterio normativo o con un requisito del caso. Las puertas bloquean el merge a main.

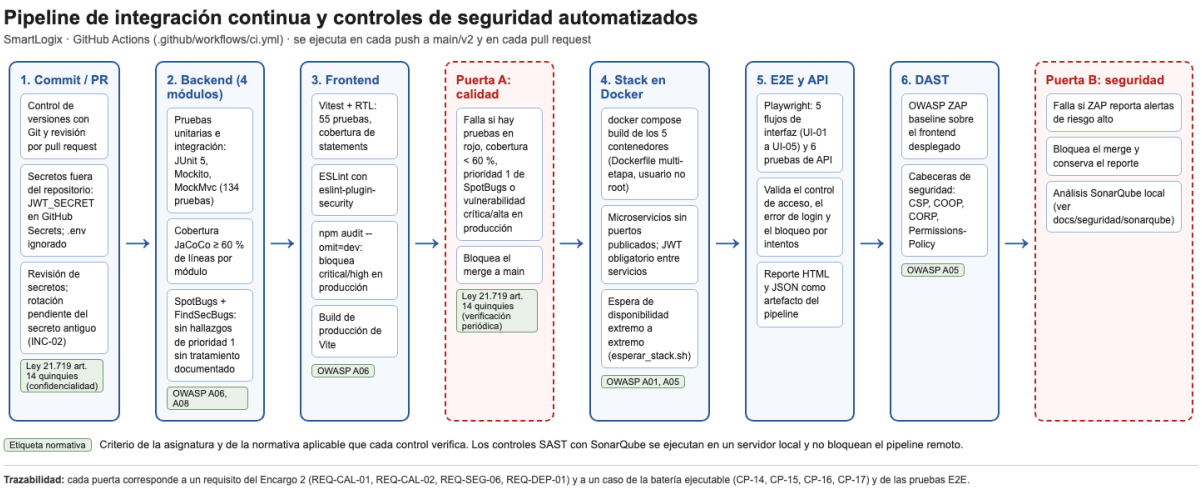


Figura 1. Pipeline de integración continua y controles automatizados (docs/seguridad/diagramas/pipeline.html).

Control	Verifica	Criterio normativo o de calidad	Bloquea si...
Pruebas backend + JaCoCo	Comportamiento y cobertura de 4 módulos	ISO/IEC 25010 (mantenibilidad)	Hay pruebas en rojo o cobertura < 60 %
SpotBugs + FindSecBugs	Patrones de seguridad de Spring y Java	OWASP A05 (configuración)	Hay hallazgos de prioridad 1 sin tratamiento
Pruebas frontend + ESLint	Comportamiento de componentes y patrones inseguros	OWASP A03 (inyección)	Hay pruebas en rojo o errores de lint
npm audit --omit=dev	Vulnerabilidades en dependencias de ejecución	OWASP A06 (componentes vulnerables)	Hay critical o high en producción
Stack Docker + E2E/API	Flujos reales con autenticación y validaciones	Ley 21.719 art. 14 quinquies (medidas de seguridad)	Falla un flujo o la API responde mal
OWASP ZAP baseline	Cabeceras, errores y configuración en ejecución	OWASP A05; Ley 21.719 art. 14 quinquies	Hay alertas de riesgo alto
SonarQube (local)	Calidad, duplicación y vulnerabilidades del código	ISO/IEC 25010; criterio de la asignatura	Gate de calidad no superado (ver INC-24)

SonarQube se ejecuta en un servidor local porque el pipeline remoto no puede acceder a localhost. Sus resultados se incluyen como evidencia del análisis, y el job de SonarQube queda preparado para activarse con los secretos del repositorio. Esa diferencia se declara en el informe.

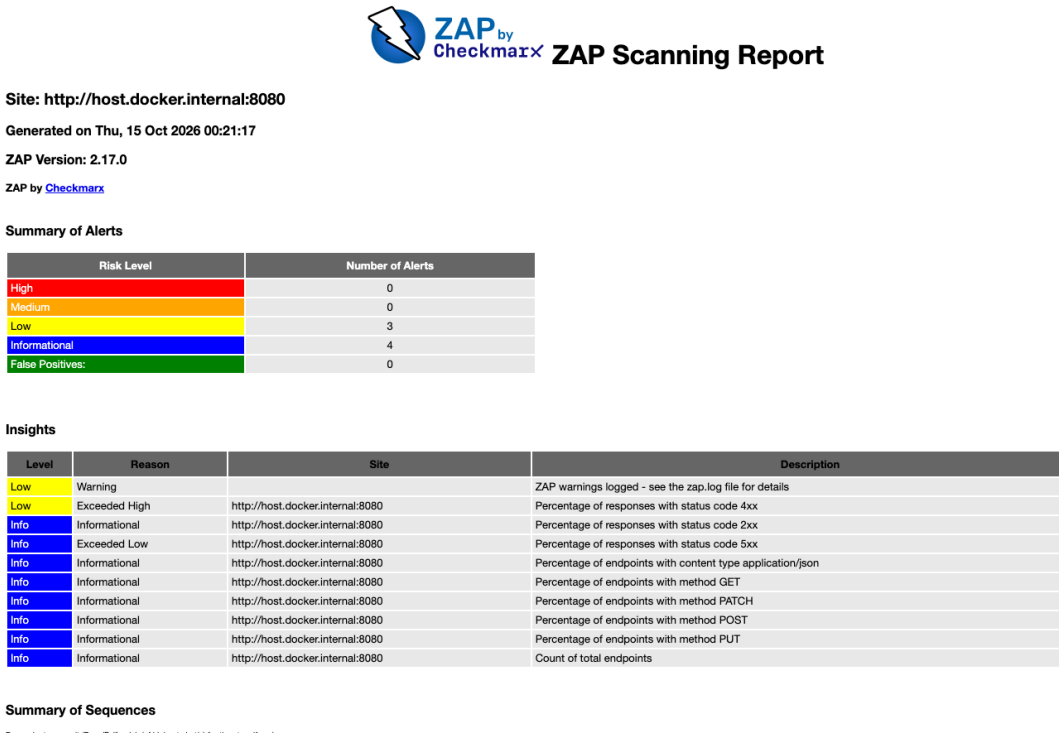
5. Resultados de las pruebas de seguridad

5.1 Análisis dinámico (OWASP ZAP)

Se ejecutó ZAP en modo baseline sobre el frontend y como escaneo de la API (OpenAPI del BFF) con un token válido inyectado. La tabla compara la versión de referencia (main) con la versión final (v2). Las cifras son instancias de alerta según el reporte JSON.

Riesgo	Frontend (main)	Frontend (v2)	API (main)	API (v2)
High	0	0	0	0
Medium	3	1	0	0
Low	19	3	17	9
Informational	8	8	660	665

Lectura de los resultados: no hay alertas de riesgo alto en ninguna versión, por lo que la puerta B del pipeline queda superada. En el frontend, las alertas de riesgo medio bajan de 3 a 1: la residual es 'style-src unsafe-inline', necesaria porque los componentes usan estilos en línea (INC-21). En la API, las alertas de baja severidad bajan de 17 a 9. Quedan respuestas 500 de baja severidad en pruebas de inyección de plantillas sobre parámetros de búsqueda; se documentan como observación.



ZAP by Checkmarx ZAP Scanning Report

Site: http://host.docker.internal:8080

Generated on Thu, 15 Oct 2026 00:21:17

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

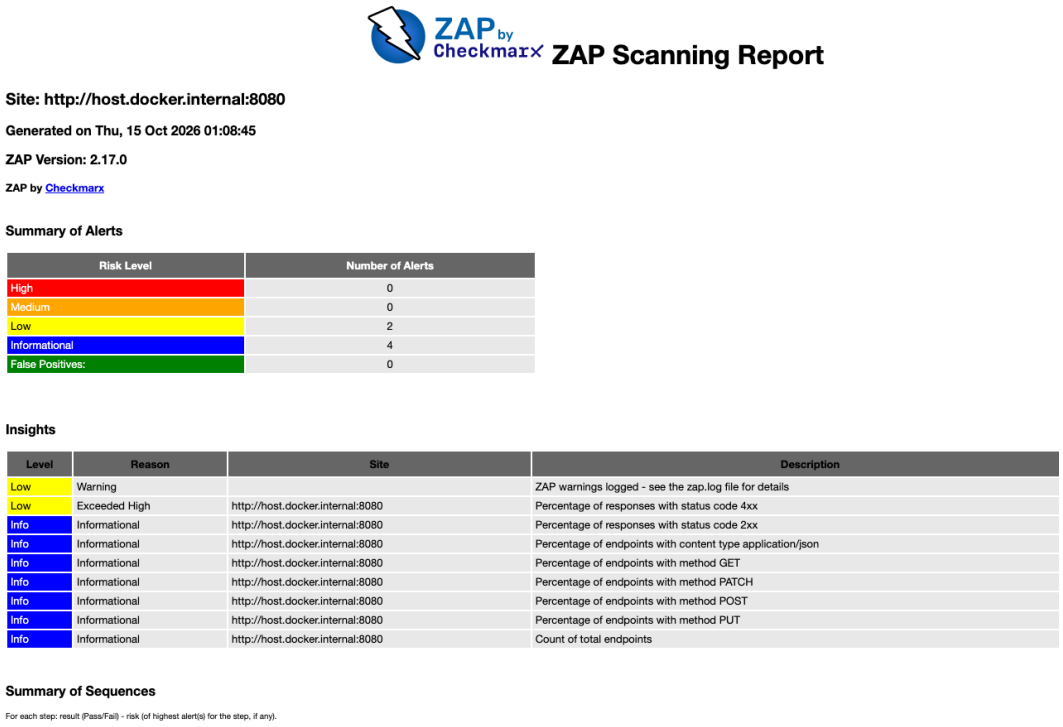
Risk Level	Number of Alerts
High	0
Medium	0
Low	3
Informational	4
False Positives:	0

Insights

Level	Reason	Site	Description
Low	Warning		ZAP warnings logged - see the zap.log file for details
Low	Exceeded High	http://host.docker.internal:8080	Percentage of responses with status code 4xx
Info	Informational	http://host.docker.internal:8080	Percentage of responses with status code 2xx
Info	Exceeded Low	http://host.docker.internal:8080	Percentage of responses with status code 5xx
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with content type application/json
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with method GET
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with method PATCH
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with method POST
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with method PUT
Info	Informational	http://host.docker.internal:8080	Count of total endpoints

Summary of Sequences

Figura 2. Reporte ZAP de la API en la versión de referencia (main).



ZAP Scanning Report

Site: http://host.docker.internal:8080

Generated on Thu, 15 Oct 2026 01:08:45

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	2
Informational	4
False Positives:	0

Insights

Level	Reason	Site	Description
Low	Warning		ZAP warnings logged - see the zap.log file for details
Low	Exceeded High	http://host.docker.internal:8080	Percentage of responses with status code 4xx
Info	Informational	http://host.docker.internal:8080	Percentage of responses with status code 2xx
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with content type application/json
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with method GET
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with method PATCH
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with method POST
Info	Informational	http://host.docker.internal:8080	Percentage of endpoints with method PUT
Info	Informational	http://host.docker.internal:8080	Count of total endpoints

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Figura 3. Reporte ZAP de la API en la versión final (v2).

5.2 Análisis estático (SAST)

Módulo	Hallazgos totales / prioridad 1 (v2)
inventory-service	13 / 0
order-service	25 / 0
shipping-service	11 / 0
bff-gateway	25 / 0

En v2 no hay hallazgos de prioridad 1 sin tratamiento. El único de prioridad 1 (SPRING_CSRF_PROTECTION_DISABLED) queda como riesgo aceptado en cada módulo, con su exclusión documentada en spotbugs-exclude.xml. La versión de referencia tenía un hallazgo de prioridad 1 en el BFF.

Métrica SonarQube	Backend (v2)	Frontend (v2)
Bugs	0	0
Vulnerabilidades abiertas	0	0
Vulnerabilidades aceptadas (WONTFIX)	4	0
Code smells	0	0
Cobertura	87.3 %	68.4 %
Duplicación (global)	14.5 %	1.1 %

Quality gate	ERROR	OK

El quality gate del backend falla por la duplicación de código en código nuevo (29.69 %). Es un hallazgo real de mantenibilidad (INC-24) y no se ocultó: la propuesta SEG-03 lo resuelve con un módulo común. Las cuatro vulnerabilidades S4502 son el mismo riesgo CSRF de cada módulo y se aceptaron en SonarQube con una justificación registrada en el issue.

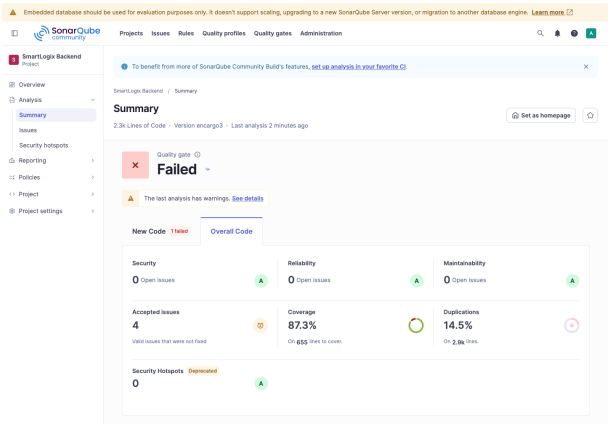


Figura 4. SonarQube, backend (vista general): gate fallido por duplicación.

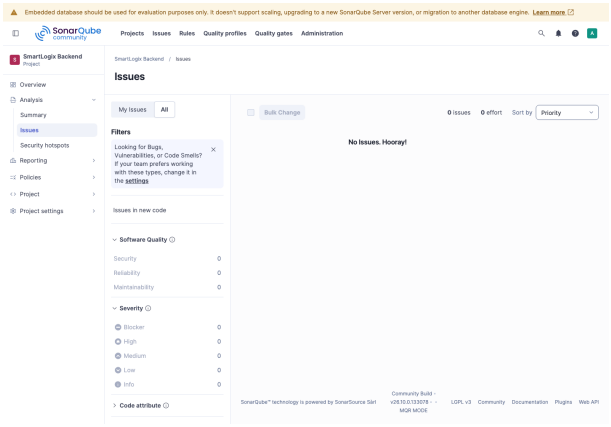


Figura 5. SonarQube, issues del backend (riesgo CSRF aceptado).

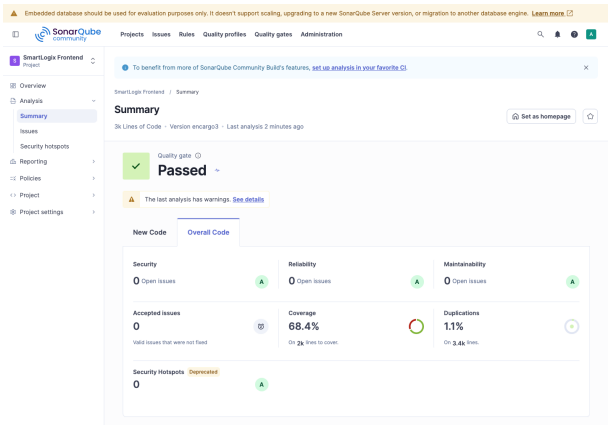


Figura 6. SonarQube, frontend: gate superado.

5.3 Análisis de dependencias (SCA)

Alcance	Crítica	Alta	Moderada	Total
Completo (main, Encargo 2)	3	13	7	23
Producción (v2): npm audit --omit=dev	0	0	2	2

Completo (v2): incluye build y test	3	6	6	15

La versión de referencia tenía 23 vulnerabilidades en total (3 críticas y 13 altas, según el Encargo 2). Las dependencias de producción quedan sin critical ni high. La moderada restante (react-router-dom 6) exige migrar a la versión 7, que es un cambio mayor; se mantiene como riesgo documentado. Las críticas y altas que quedan son de vitest, vite y tailwind, usados solo al compilar y probar (INC-09).

5.4 Batería de casos de prueba

La batería contiene 23 casos. Se ejecutó en la versión de referencia (main) y en la final (v2) con el mismo script (docs/pruebas/ejecutar_casos.py). Cada caso registra requisito, datos de entrada, resultado esperado, criterio de aceptación, criticidad y evidencia. Las tarjetas de cada caso están en el anexo A.

Caso	Descripción	Requisito	Base	Final	Criticidad
CP-01	Autenticación JWT y acceso a endpoints protegidos	REQ-SEG-01	PASA	PASA	CRITICO
CP-02	Rechazo de credenciales inválidas y registro duplicado	REQ-SEG-01	PASA	PASA	MODERADO
CP-03	Descuento según tipo de orden (Factory Method)	REQ-FUNC-02	PASA	PASA	GRAVE
CP-04	Rechazo de tipo de orden no soportado	REQ-FUNC-02	PASA	PASA	MODERADO
CP-05	Fallback del Circuit Breaker ante fallo del carrier	REQ-RES-01	PASA	PASA	GRAVE
CP-06	Búsqueda y filtrado de productos	REQ-FUNC-03	PASA	PASA	MODERADO
CP-07	Validación de entradas en servicios de negocio	REQ-VAL-01	PASA	PASA	GRAVE
CP-08	Validación de entradas a través del gateway (BFF)	REQ-VAL-01	PASA	PASA	MODERADO
CP-09	Tiempos de respuesta (lectura y escritura)	REQ-NFR-01	PASA	PASA	MODERADO
CP-10	Protección de contraseñas en almacenamiento	REQ-SEG-02	PASA	PASA	GRAVE
CP-11	Errores sin exposición de detalles internos	REQ-SEG-03	PASA	PASA	MODERADO
CP-12	Restricción de orígenes CORS	REQ-SEG-04	PASA	PASA	MODERADO
CP-13	Secreto de firma JWT fuera del código fuente	REQ-SEG-05	PASA	PASA	GRAVE
CP-14	Cobertura de líneas JaCoCo >= 60 %	REQ-CAL-01	PASA	PASA	MODERADO
CP-15	Sin hallazgos de prioridad 1 en análisis estático (SpotBugs	REQ-CAL-02	FALLA	PASA	MODERADO
CP-16	Dependencias del frontend sin vulnerabilidades críticas o al	REQ-SEG-06	FALLA	FALLA	GRAVE
CP-17	Despliegue reproducible en contenedores Docker	REQ-DEP-01	PASA	PASA	MODERADO
CP-18	Aislamiento de los microservicios de negocio	REQ-SEG-07	PASA	PASA	CRITICO
CP-19	Alta de producto con persistencia y asignación de ID	REQ-FUNC-04	PASA	PASA	GRAVE
CP-20	Microservicio rechaza peticiones sin token desde la red inte	REQ-SEG-08	FALLA	PASA	GRAVE
CP-21	Bloqueo temporal tras intentos fallidos de inicio de sesión	REQ-SEG-09	FALLA	PASA	MODERADO

CP-22	Actualización de estado de pedido a través del BFF	REQ-FUNC-05	FALLA	PASA	GRAVE
CP-23	Parámetro obligatorio ausente devuelve 400	REQ-VAL-02	FALLA	PASA	MODERADO

Notas de la comparación. CP-15 falla en la base solo porque la rama v2 regeneró los reportes de SpotBugs; en la ejecución final pasa con los reportes actualizados. CP-16 es el riesgo residual documentado (INC-09). La versión de referencia también se evaluó con los casos CP-20 a CP-23, añadidos en este encargo.

6. Resultados de las pruebas automatizadas

6.1 Suites automatizadas y cobertura

Suite	Pruebas	Cobertura	Alcance
inventory-service	41	95.1 %	142 líneas
order-service	29	93.8 %	161 líneas
shipping-service	28	92.8 %	139 líneas
bff-gateway	36	77.1 %	214 líneas
Total backend	134	—	—
Frontend (Vitest + RTL)	55	68,5 % statements	13 archivos de prueba
E2E y API (Playwright)	11	—	5 flujos de interfaz y 6 de API

Cada módulo supera el umbral de 60 % de líneas. El gate de JaCoCo está activo porque el argLine se corrigió (INC-07). Las pruebas nuevas de seguridad son de integración con el contexto completo de Spring, con su propia base H2 para evitar colisiones entre contextos (SecurityIntegrationTest).

6.2 Pruebas E2E y de API

Prueba (versión v2)	Resultado	Duración
UI-01 acceso a una ruta protegida sin sesión redirige a login	PASSED	0.25 s
UI-02 registro de un usuario nuevo muestra confirmación	PASSED	1.28 s
UI-03 credenciales incorrectas muestran un mensaje de error	PASSED	1.50 s
UI-04 inicio de sesión correcto lleva a la página principal	PASSED	1.55 s
UI-05 el catálogo muestra los productos obtenidos del BFF	PASSED	1.46 s
API-01 endpoint protegido sin token responde 401	PASSED	0.01 s
API-02 listado de productos con token responde 200 con el catálogo	PASSED	0.01 s
API-03 tipo de orden no soportado responde 400 con mensaje controlado	PASSED	0.01 s
API-04 actualizar estado de un pedido a través del BFF responde 200	PASSED	0.02 s
API-05 parámetro obligatorio ausente responde 400 y no 500	PASSED	0.00 s
API-06 con token inválido se rechaza el acceso	PASSED	0.00 s

Resultado de la ejecución final: 11 de 11 pruebas aprobadas, 0 fallidas y 0 inestables, en 6.8 s. En la rama main, la misma suite obtuvo 8 de 11: UI-03 falló por INC-18, API-04 por INC-19 (503) y API-05 por INC-20 (500). La transcripción de esa ejecución está en docs/evidencias/encargo3/e2e-linea-base-main.txt.

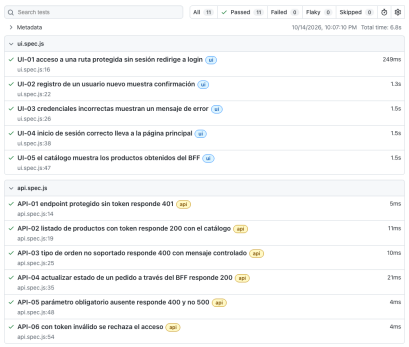


Figura 7. Reporte HTML de Playwright (ejecución final).

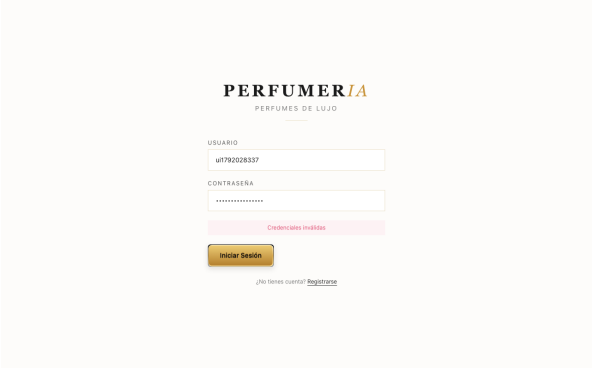


Figura 8. UI-03 en v2: se muestra el error y se conserva el usuario (INC-18 cerrado).

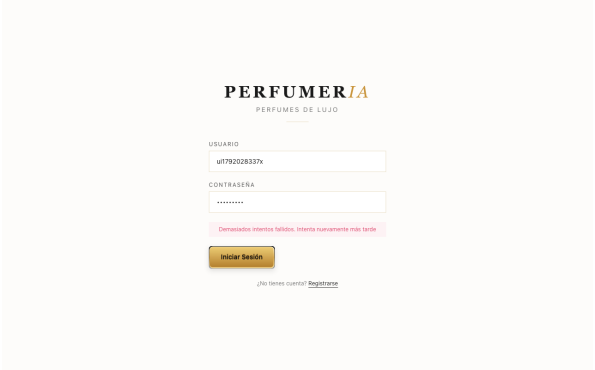


Figura 9. UI-06 en v2: bloqueo temporal tras cinco intentos fallidos (INC-13).

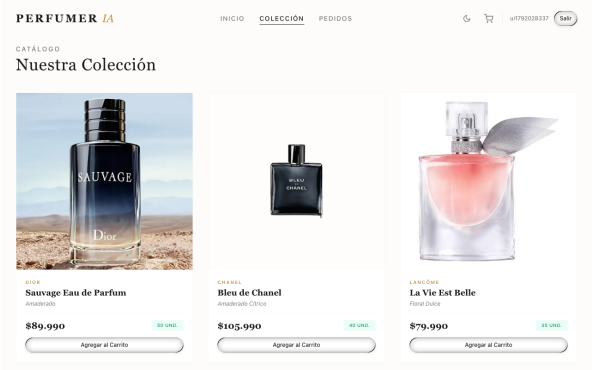


Figura 10. UI-05: catálogo cargado desde el BFF con token.

6.3 Pipeline de integración continua

El workflow .github/workflows/ci.yml se ejecutó en la rama v2 y en el pull request #1. Todos los jobs terminaron con éxito en la última ejecución (ejecución 38665699298, 9 minutos de duración total). La primera ejecución del DAST falló por permisos de escritura del directorio de reportes; el registro del job lo mostró y se corrigió en un commit posterior (incidente de pipeline, no de producto).

Job	Resultado	Duración
Backend (bff-gateway)	Éxito	41 s
Backend (inventory-service)	Éxito	45 s
Backend (order-service)	Éxito	45 s
Backend (shipping-service)	Éxito	42 s
Frontend	Éxito	28 s
E2E y API sobre Docker	Éxito	5 min 15 s
DAST (OWASP ZAP baseline)	Éxito	2 min 56 s

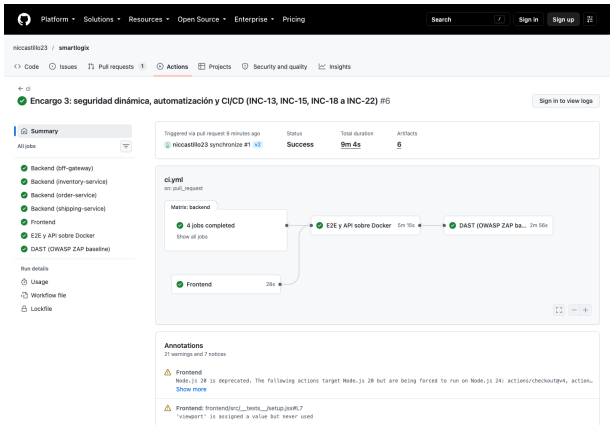


Figura 11. Ejecución del pipeline en GitHub Actions: todos los jobs en verde.

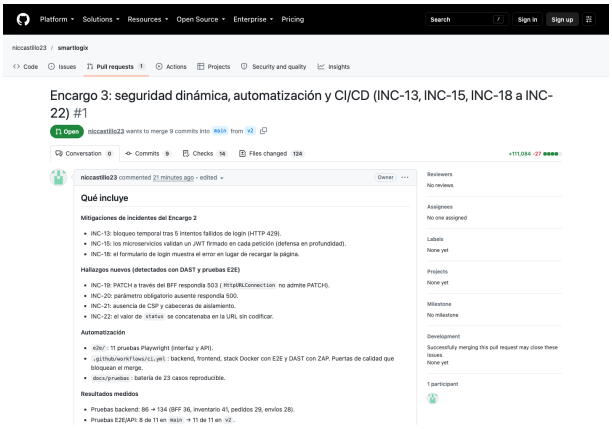


Figura 12. Pull request #1 hacia main con el pipeline ejecutado.

6.4 Eficiencia de la automatización

La automatización cubre 134 pruebas de backend, 55 de frontend, 11 de E2E y API, y la batería de 23 casos. El tiempo de ejecución automatizado de la suite E2E y de API es de 6,8 segundos. Cada job de backend del pipeline tarda entre 41 y 49 segundos, incluida la preparación del entorno. El ahorro frente a la verificación manual no se midió con cronómetro: esa comparación requiere ejecutar los mismos casos a mano y registrar su tiempo, y no se hizo. La eficiencia se justifica en la repetibilidad: los mismos casos se ejecutan en cada PR y detectan regresiones sin intervención.

6.5 Regresión: qué prueba detecta cada defecto corregido

Para cada defecto corregido se identifica la prueba que lo detecta. El criterio es que la prueba falle sobre la versión de referencia y pase sobre la versión final. Así, la corrección queda protegida frente a cambios futuros.

Incidente	Prueba que lo detecta	Comportamiento en main	Comportamiento en v2
INC-13	CP-21; AuthServiceTest (bloqueo y reseteo)	Falla sin bloqueo	Pasa
INC-15	CP-20; SecurityIntegrationTest x3	CP-20 falla (200 sin token)	Pasa
INC-18	UI-03 (Playwright)	Falla: el error no aparece	Pasa
INC-19	API-04; CP-22	503 en PATCH	Pasa
INC-20	API-05; CP-23	500 con parámetro ausente	Pasa
INC-22	StatusUrlEncodingTest x2	No ejecutada sobre main; el código de main concatena el valor (verificado por lectura del código)	Pasa
INC-16	CP-19; ProductSeedIdIntegrationTest	Falla: HTTP 500 en el alta	Pasa
INC-21	ZAP baseline frontend (DAST)	Medio: 3 alertas	Medio: 1 alerta (residual)

7. Incidentes identificados, criticidad y propuestas de mitigación

La escala de criticidad es la de la plantilla: CRÍTICO (compromete credenciales o datos, o impide el uso; remediación inmediata), GRAVE (afecta funcionalidades importantes o la cadena de desarrollo; corrección urgente), MODERADO (afecta casos específicos; próxima iteración) y LEVE (apariencia o texto). El estado distingue entre cerrado, mitigado con residual, riesgo aceptado y abierto.

ID	Caso / origen	Requisito	Descripción	Ubicación	Criticidad	Estado	Tratamiento y evidencia
INC-01	CP-10	REQ-SEG-02	Contraseñas almacenadas en texto plano y comparadas por igualdad.	bff-gateway AuthService (base: líneas 17, 24, 39)	CRÍTICO	CERRADO	Hash BCrypt. Prueba: AuthServiceTest.register_shouldStoreBcryptHashNotPlaintext; CP-10.
INC-02	CP-13	REQ-SEG-05	Secreto de firma JWT literal en application.yml, presente en el historial de git.	bff-gateway application.yml (base: línea 17); historial anterior a 2d7685f	CRÍTICO	CERRADO EN CÓDIGO; ROTACIÓN PENDIENTE	Secreto desde ${JWT_SECRET}. Debe rotarse: el valor del historial se considera expuesto.
INC-15	CP-20	REQ-SEG-07	Los microservicios respondían sin token a cualquier petición de la red interna.	inventory, order y shipping: sin Spring Security; puertos publicados (base)	CRÍTICO	CERRADO	JwtAuthFilter y SecurityConfig en cada servicio; SecurityIntegrationTest (3 pruebas por servicio); CP-20 (401 desde la red interna).
INC-22	Revisión DAST	REQ-FUNC-05	El valor de status se concatenaba en la URL: '&', '#' o llaves alteraban la petición al microservicio.	bff-gateway OrderProxyService y ShippingProxyService (base)	GRAVE	CERRADO	Construcción de URI codificada. StatusUrlEncodingTest (2 pruebas) y PATCH de prueba con 'SHIPPED&admin=true' (200, sin parámetros inyectados).
INC-16	CP-19	REQ-FUNC-04	Alta de productos con HTTP 500: la secuencia de IDs no avanzaba tras la semilla.	inventory-service data.sql (base)	GRAVE	CERRADO	ALTER TABLE ... RESTART WITH. ProductSeedIdIntegrationTest (falla sin la corrección); CP-19.
INC-19	API-04 / CP-22	REQ-FUNC-05	PATCH a través del BFF respondía 503: RestTemplate con HttpURLConnection no admite PATCH.	bff-gateway RestTemplateConfig (base)	GRAVE	CERRADO	JdkClientHttpRequestFactory. API-04 y CP-22 pasan.
INC-13	CP-21	REQ-SEG-09	Sin límite de intentos de login: fuerza bruta sin freno.	bff-gateway AuthService/AuthController (base)	MODERADO	CERRADO	Bloqueo de 15 min tras 5 fallos por usuario (HTTP 429). CP-21; AuthServiceTest (2 pruebas nuevas).
INC-18	UI-03	REQ-SEG-01	El login con credenciales incorrectas recargaba la página y no mostraba el error.	frontend api.js (interceptor 401 sin excepción para /auth/)	MODERADO	CERRADO	Excepción para /auth/*. UI-03 pasa: se muestra 'Credenciales inválidas' y se conserva el usuario.
INC-20	API-05 / CP-23	REQ-VAL-02	Parámetro obligatorio ausente devolvía HTTP 500.	GlobalExceptionHandler de los cuatro módulos (base)	MODERADO	CERRADO	MissingServletRequestParameterException mapeada a 400. API-05 y CP-23.
INC-21	DAST frontend	REQ-SEG-10	Sin Content-Security-Policy ni cabeceras de aislamiento; versión del servidor expuesta.	frontend/nginx.conf (base)	MODERADO	MITIGADO (residual)	CSP, COOP, CORP, Permissions-Policy y server_tokens off. Residual: 'unsafe-inline' en estilos (componentes con estilos en línea).
INC-08	CP-15	REQ-CAL-02	SpotBugs prioridad 1: SPRING_CSRF_PROTECTION_DISABLED en la configuración de seguridad.	SecurityConfig de los cuatro módulos	MODERADO	RIESGO ACEPTADO	API stateless con Bearer, sin cookies. Exclusión documentada en cada módulo y 4 issues S4502 aceptados en SonarQube con justificación.
INC-09	CP-16	REQ-SEG-06	Vulnerabilidades en dependencias de desarrollo del frontend (vitest, vite, tailwind).	frontend/package.json (devDependencies)	GRAVE	ABIERTO (residual)	Producción sin high ni critical (npm audit --omit=dev). Quedan 3 críticas y 6 altas en build/test; corregir exige saltos mayores.
INC-24	SonarQube	REQ-CAL-03	Duplicación de código entre microservicios (SecurityConfig, JwtAuthFilter, manejador de errores y pruebas): el quality gate falla con 29,7 % de duplicación en código nuevo.	inventory, order, shipping y bff-gateway	MODERADO	ABIERTO	Extraer un módulo común (librería de seguridad y errores). Ver propuesta SEG-03.
INC-12	Revisión de código	REQ-SEG-01	El token JWT se guarda en localStorage, accesible desde scripts de la página.	frontend/src/context/AuthContext.jsx	MODERADO	ABIERTO	Cookie HttpOnly con SameSite y protección CSRF reactivada. Ver propuesta SEG-02.
INC-14	Revisión visual	REQ-CAL-01	La pantalla de inicio se presenta como 'PERFUMERÍA' y el título HTML como	frontend (captura UI-01 a UI-05)	LEVE	ABIERTO	Unificar el nombre comercial.

			'SmartLogix'.				
INC-03	CP-11	REQ-SEG-03	Excepciones de conversión devolvían 404 con el mensaje interno del framework.	GlobalExceptionHandler (base)	MODERADO	CERRADO	Respuesta genérica sin detalle técnico. CP-11.
INC-04	CP-02	REQ-SEG-01	Credenciales inválidas y registro duplicado respondían 403.	bff-gateway (base)	MODERADO	CERRADO	401 y 409. CP-02.
INC-05	CP-04	REQ-FUNC-02	Tipo de orden no soportado respondía 404.	order-service OrderService (base)	MODERADO	CERRADO	400 con mensaje controlado. CP-04.
INC-07	CP-14	REQ-CAL-01	El gate de JaCoCo no se ejecutaba: el argLine de Surefire sustituía al agente.	pom.xml de los cuatro módulos (base)	MODERADO	CERRADO	argLine = @{argLine}. CP-14 con cobertura medida por módulo.
INC-23	Revisión de código	REQ-SEG-03	Los logs del BFF registraban el mensaje de excepción (riesgo de inyección CRLF en logs).	bff-gateway GlobalExceptionHandler (base)	LEVE	CERRADO	Se registra solo el código HTTP o un texto constante. SpotBugs CRLF: 0.
INC-25	SonarQube	REQ-CAL-03	Frontend: 2 bugs (promesas sin manejo de error) y 2 code smells (regex super-lineal, isNaN).	OrderStatus.jsx, ProductCatalog.jsx, formatPrice.js	LEVE	CERRADO	Manejo de error en las promesas y expresión sin regex. SonarQube frontend: 0 bugs y 0 smells.
INC-06	CP-08	REQ-VAL-01	Un error 400 de validación llegaba al cliente como 403 a través del BFF.	bff-gateway (base; causa probable)	MODERADO	CERRADO	Propagación de errores 4xx del servicio de destino. CP-08.
INC-10	CP-17	REQ-DEP-01	Sin Dockerfile ni docker-compose.yml.	Raíz del repositorio (base)	MODERADO	CERRADO	Imágenes multi-etapa y Compose con JWT_SECRET obligatorio. CP-17.
INC-11	Revisión documental	REQ-CAL-01	Cifras de cobertura de la Ev1 no reproducibles.	Informe de la Ev1	MODERADO	CERRADO	Cifras medidas y corregidas en el informe del Encargo 2.

Resumen: 24 incidentes (crítico: 3, grave: 4, moderado: 14, leve: 3). Cerrados: 18. Abiertos, mitigados con residual o con riesgo aceptado: 6. Los incidentes críticos INC-01, INC-02 y INC-15 están cerrados en el código; INC-02 requiere además rotar el secreto, porque el valor original sigue en el historial de git.

7.1 Propuestas de mitigación para los defectos abiertos

ID	NOMBRE	IMPACTO
SEG-01	Rotación del secreto JWT y limpieza del historial	CRÍTICO (por historial público)

PROPUESTA DE REMEDIACIÓN
Generar un nuevo JWT_SECRET en GitHub Secrets y en el .env de cada entorno, invalidar los tokens emitidos con el secreto anterior y reescribir el historial con git filter-repo (operación destructiva que requiere autorización expresa del equipo).

ID	NOMBRE	IMPACTO
SEG-02	Mover el token a una cookie HttpOnly	MODERADO (INC-12)

PROPUESTA DE REMEDIACIÓN
Emitir el JWT en una cookie HttpOnly, Secure y SameSite=Strict; el frontend deja de leer el token. Reactivar la protección CSRF con token sincronizado y retirar la exclusión de SpotBugs. Impacto: cambio de la autenticación del frontend y pruebas E2E nuevas.

ID	NOMBRE	IMPACTO
SEG-03	Módulo común de seguridad y manejo de errores	MODERADO (INC-24)

PROPUESTA DE REMEDIACIÓN
Extraer SecurityConfig, JwtAuthFilter, GlobalExceptionHandler y la plantilla de pruebas de integración a una librería compartida (common-security). Reduce la duplicación en los cuatro módulos, cumple el gate de SonarQube y concentra las correcciones de seguridad en un solo lugar.

ID	NOMBRE	IMPACTO
SEG-04	Eliminar 'unsafe-inline' de la CSP	MODERADO (INC-21)

PROPUESTA DE REMEDIACIÓN
Sustituir los estilos en línea por clases de Tailwind y usar hashes o nonces para los estilos generados. Verificar con ZAP y con las pruebas E2E, porque framer-motion aplica estilos dinámicos.

ID	NOMBRE	IMPACTO
SEG-05	Migración de herramientas de build y react-router	GRAVE (INC-09)

PROPUESTA DE REMEDIACIÓN
Actualizar vitest, vite y tailwind a versiones mayores compatibles, y evaluar react-router 7. Revalidar la cobertura con Vitest 5 antes de adoptarla: la versión 5 probada redujo la cobertura medida de 68,5 % a 51,8 %.

ID	NOMBRE	IMPACTO
SEG-06	Protección de la autenticación	MODERADO (INC-13 residual)

PROPUESTA DE REMEDIACIÓN
El bloqueo por usuario puede usarse para negar el acceso a una cuenta legítima. Añadir límite por IP y un tiempo

de bloqueo creciente, y notificar al titular de los intentos de acceso.

ID	NOMBRE	IMPACTO
SEG-07	Revisión de la clasificación de SonarQube	MODERADO (INC-08)

PROPUESTA DE REMEDIACIÓN
Revisar cada cuatro meses los cuatro issues S4502 aceptados, para confirmar que la API sigue siendo stateless y sin cookies.

8. Plan de mantenimiento y preservación de la calidad, la seguridad y la normativa

El plan asegura que las mejoras de este encargo se mantengan en versiones futuras. Define qué se revisa, con qué frecuencia, quién lo hace y qué criterio determina si el cambio puede avanzar. Los responsables son los integrantes del grupo, con asignación rotativa, según la práctica de la asignatura.

Actividad	Frecuencia	Responsable	Criterio de éxito
Regresión completa: pruebas unitarias, batería de casos, E2E y DAST	En cada PR y antes de cada versión	Quien abre el PR	Pipeline en verde y ninguna prueba de regresión fallida
Auditoría de dependencias (npm audit --omit=dev y completo) y Maven	Cada semana (Dependabot) y antes de cada versión	Responsable de dependencias	Sin critical ni high en producción
Revisión de SpotBugs y SonarQube	Cada PR y mensual	Revisor de calidad	Prioridad 1 tratada; gate superado o deuda registrada
Revisión de cabeceras, CORS y CSP con ZAP	Mensual y ante cambios en nginx o en la configuración de seguridad	Responsable de seguridad	Sin alertas de riesgo alto; residuales en el registro
Rotación de JWT_SECRET	Cada 90 días y tras cualquier sospecha	Responsable de despliegue	Secreto nuevo en GitHub Secrets; tokens antiguos invalidados
Revisión normativa (Ley 21.719 y reglamentos de la Agencia)	Cada seis meses y al publicarse cada reglamento	Responsable de cumplimiento	Registro de la revisión y de las obligaciones pendientes
Registro y revisión de incidentes de seguridad	Continuo; revisión mensual	Responsable de seguridad	Todo incidente con criticidad, causa y corrección
Verificación de copias y de retención de logs	Mensual	Responsable de operación	Logs sin datos personales innecesarios
Gestión de la deuda técnica (SEG-03, INC-24)	Trimestral	Equipo	Duplicación del gate por debajo del umbral

8.1 Política de regresión y de cambios

- **Todo defecto corregido tiene una prueba:** la prueba debe fallar antes de la corrección y pasar después (sección 6.5).
- **Ningún cambio de seguridad se fusiona sin pruebas:** autenticación, cabeceras y configuración requieren prueba automatizada o caso en la batería.
- **Las excepciones se documentan:** un riesgo aceptado se registra en el informe, en SonarQube y en el archivo de exclusión, con su justificación y revisión periódica.

- **Las pruebas nuevas no dependen del orden:** cada contexto de prueba usa su propia base y cada prueba de usuario crea un usuario único.

8.2 Preservación de los aspectos normativos y legales

La Ley 21.719 reforma la Ley 19.628 y crea la Agencia de Protección de Datos Personales. Según la Biblioteca del Congreso Nacional, entra en vigencia el 1 de diciembre de 2026; en septiembre de 2026 el Gobierno propuso postergarla al 1 de diciembre de 2027. Este informe no puede confirmar si la propuesta se aprobó: debe verificarse en bcn.cl antes de la entrega y antes de tratar datos reales.

Obligación (ley reformada)	Estado en SmartLogix	Acción propuesta
Medidas de seguridad apropiadas (art. 14 quinquies): cifrado, confidencialidad e integridad	Parcial: BCrypt, JWT entre servicios, CSP, bloqueo de intentos. Sin HTTPS en el entorno de demostración y con H2 en memoria.	TLS terminado en el gateway; base de datos persistente cifrada en reposo (propuesta del Encargo 2, R6).
Reporte de vulneraciones a la Agencia (art. 14 sexies)	No existe procedimiento ni registro de vulneraciones.	Procedimiento de respuesta a incidentes con plazos, responsable y plantilla de notificación.
Derechos del titular: acceso, rectificación, supresión, oposición, portabilidad (arts. 4 y 9)	No hay endpoints para ejercerlos.	Endpoints de consulta, corrección, supresión y exportación en formato estructurado.
Consentimiento y transparencia (arts. 12 y 14 ter)	El registro no solicita consentimiento ni publica política de privacidad.	Casilla de consentimiento en el registro y política de privacidad publicada.
Minimización y conservación de datos (principio de proporcionalidad)	Se guardan nombre, correo y dirección de envío sin plazo de conservación.	Política de retención y purga; logs sin datos personales.

9. Análisis ético y normativo de los resultados

9.1 Análisis ético

- **Independencia de la evaluación:** el equipo evalúa código propio. Para reducir el sesgo, los resultados esperados se fijaron antes de ejecutar y las discrepancias se registran como incidentes, no como cambios silenciosos de la expectativa.
- **Resultados desfavorables incluidos:** se reportan los fallos de la versión de referencia, el riesgo residual (INC-09), la cobertura que bajó con Vitest 5 y el quality gate de SonarQube que no se supera. Ocultar esos datos sería engañoso.
- **Divulgación responsable:** las vulnerabilidades se describen con su ubicación para permitir la corrección. El secreto y las contraseñas no se reproducen; los tokens se redactan antes de guardar cualquier evidencia.
- **Alcance autorizado:** todas las pruebas dinámicas se ejecutaron sobre el stack local y sobre el repositorio propio, en línea con la autorización de la asignatura. No se apuntó a sistemas de terceros.
- **Datos personales:** los usuarios de prueba son sintéticos y las ejecuciones usan datos de semilla. Aun así, el análisis trata la protección de datos como un requisito, porque el diseño permitiría tratar datos reales sin las medidas faltantes.

- **Responsabilidad sobre los riesgos aceptados:** el riesgo CSRF y el 'unsafe-inline' se aceptan con una razón técnica concreta y una fecha de revisión, para que no se conviertan en deuda olvidada.

9.2 Análisis normativo

Desde la perspectiva normativa, los controles técnicos avanzan de forma clara: la autenticación, el secreto, el bloqueo y las cabeceras reducen el riesgo de acceso indebido. Las brechas que quedan son de gobierno de datos más que de código: derechos del titular, consentimiento, respuesta a incidentes y conservación. Esas brechas deben cerrarse antes del tratamiento de datos reales. El cumplimiento de la Ley 21.719 depende además de la fecha de entrada en vigencia, que debe confirmarse, y de los reglamentos que dicte la Agencia.

10. Conclusiones y propuestas para preservar la calidad

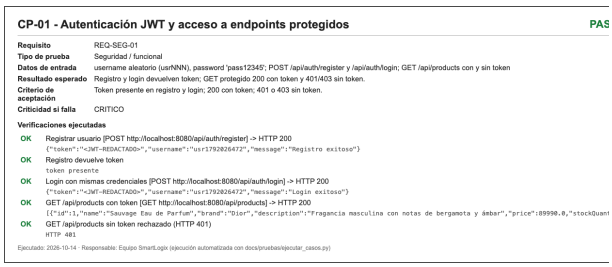
La versión de referencia aprobó 17 de 23 casos de la batería, y la versión final aprueba 22 de 23. Las 11 pruebas E2E y de API pasan en la versión final (8 de 11 en la de referencia). Los incidentes críticos de credenciales, secreto y exposición de microservicios quedan cerrados en el código. Se identificaron además cuatro defectos nuevos mediante el análisis dinámico y las pruebas de interfaz (INC-19 a INC-22), que también se corrigieron.

Quedan pendientes el secreto del historial (SEG-01), la duplicación entre módulos (SEG-03) y las dependencias de build (SEG-05). Son los puntos que más afectan a la calidad sostenible del proyecto.

- R1. Ejecutar el pipeline en cada PR y no fusionar con puertas en rojo (sección 4).
- R2. Cumplir el plan de mantenimiento de la sección 8, con responsables asignados y registro de cada revisión.
- R3. Ejecutar la batería de casos y la suite E2E después de cada cambio de seguridad, y tratar toda regresión como incidente.
- R4. Confirmar la fecha de vigencia de la Ley 21.719 y completar las obligaciones de la sección 8.2 antes de tratar datos reales.
- R5. Activar el job de SonarQube en el pipeline remoto con un servidor accesible y los secretos del repositorio.
- R6. Medir el tiempo de la verificación manual para completar la comparación de eficiencia (sección 6.4).

Anexo A. Tarjetas de los casos de prueba (ejecución final)

Cada tarjeta muestra el requisito, los datos de entrada, el resultado esperado, el criterio, la criticidad y cada verificación con su resultado observado. Se generan desde docs/evidencias/v2-final/resultados.json.



CP-01 - Autenticación JWT y acceso a endpoints protegidos PAS

Requisito	REQ-SEG-01
Tipo de prueba	Seguridad / funcional
Datos de entrada	username aleatorio (usrNNN), password 'pass12345'; POST /api/auth/register y /api/auth/login; GET /api/products con y sin token
Resultado esperado	Registro y login devuelven token; GET protegido 200 con token y 401/403 sin token.
Criterio de aceptación	Token presente en registro y login; 200 con token; 401 o 403 sin token.
Criticidad si falla	CRITICO

Verificaciones ejecutadas

- OK Registrar usuario [POST http://localhost:8080/api/auth/register] -> HTTP 200
  `{"token":"<JWT-REDACTADO>","username":"usr1792026472","message":"Registro exitoso"}`
- OK Registro devuelve token
  `token presente`
- OK Login con mismas credenciales [POST http://localhost:8080/api/auth/login] -> HTTP 200
  `{"token":"<JWT-REDACTADO>","username":"usr1792026472","message":"Login exitoso"}`
- OK GET /api/products con token [GET http://localhost:8080/api/products] -> HTTP 200
  `[{"id":1,"name":"Sauvage Eau de Parfum","brand":"Dior","description":"Fragancia masculina con notas de bergamota y ámbar","price":89990.0,"stockQuant`
- OK GET /api/products sin token rechazado (HTTP 401)
  `HTTP 401`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-01: Autenticación JWT y acceso a endpoints protegidos



CP-02 - Rechazo de credenciales inválidas y registro duplicado PAS

Requisito	REQ-SEG-01
Tipo de prueba	Seguridad / negativo
Datos de entrada	username del CP-01, password incorrecta 'clave-incorrecta'; registro duplicado con la misma cuenta
Resultado esperado	Login incorrecto: HTTP 401. Registro duplicado: HTTP 409.
Criterio de aceptación	Códigos 401 y 409 con mensaje controlado, sin token.
Criticidad si falla	MODERADO

Verificaciones ejecutadas

- OK Login con contraseña incorrecta [POST http://localhost:8080/api/auth/login] -> HTTP 401
  `{"error":"Unauthorized","status":401,"timestamp":"2026-10-15T01:07:52.876611259","message":"Credenciales inválidas"}`
- OK Registro de usuario ya existente [POST http://localhost:8080/api/auth/register] -> HTTP 409
  `{"error":"Conflict","status":409,"timestamp":"2026-10-15T01:07:52.932861592","message":"El usuario ya existe"}`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-02: Rechazo de credenciales inválidas y registro duplicado



CP-03 - Descuento según tipo de orden (Factory Method) PAS

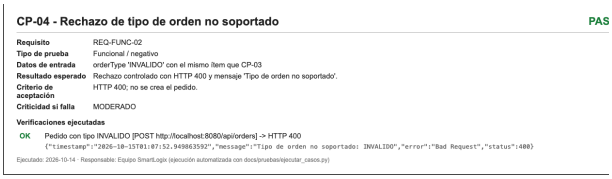
Requisito	REQ-FUNC-02
Tipo de prueba	Funcional / caja negra
Datos de entrada	customerId 1, email test@test.com, ítem productId 1 'Sauvage' cantidad 2 precio 100; orderType WEB y MARKETPLACE
Resultado esperado	WEB: 5 %, total 190,00, STANDARD. MARKETPLACE: 10 %, total 180,00, EXPRESS.
Criterio de aceptación	Descuento y total exactos; método de envío correspondiente.
Criticidad si falla	GRAVE

Verificaciones ejecutadas

- OK Pedido WEB: descuento=5.0, total=190.0, envio=STANDARD
  `HTTP 201; descuento=5.0, total=190.0, envio=STANDARD`
- OK Pedido MARKETPLACE: descuento=10.0, total=180.0, envio=EXPRESS
  `HTTP 201; descuento=10.0, total=180.0, envio=EXPRESS`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-03: Descuento según tipo de orden (Factory Method)



CP-04 - Rechazo de tipo de orden no soportado PAS

Requisito	REQ-FUNC-02
Tipo de prueba	Funcional / negativo
Datos de entrada	orderType 'INVALIDO' con el mismo ítem que CP-03
Resultado esperado	Rechazo controlado con HTTP 400 y mensaje 'Tipo de orden no soportado'.
Criterio de aceptación	HTTP 400; no se crea el pedido.
Criticidad si falla	MODERADO

Verificaciones ejecutadas

- OK Pedido con tipo INVALIDO [POST http://localhost:8080/api/orders] -> HTTP 400
  `{"timestamp":"2026-10-15T01:07:52.949863592","message":"Tipo de orden no soportado: INVALIDO","error":"Bad Request","status":400}`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-04: Rechazo de tipo de orden no soportado



CP-05 - Fallback del Circuit Breaker ante fallo del carrier PAS

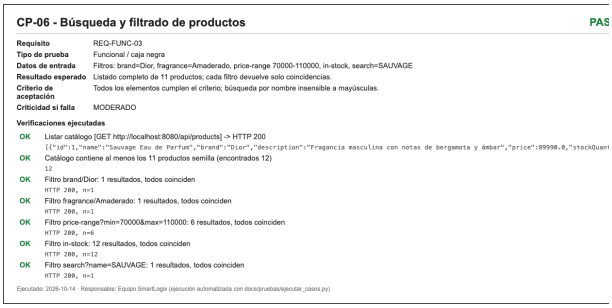
Requisito	REQ-RES-01
Tipo de prueba	Resiliencia
Datos de entrada	orderId 999, address 'Calle 123', city 'Santiago', country 'Chile'; carrier 'DHL' y 'FAIL_CARRIER'
Resultado esperado	DHL: TRK-, sin fallback, entrega +3 días. FAIL_CARRIER: FBK-, fallback=true, entrega +7 días, HTTP 201.
Criterio de aceptación	Ambos casos devuelven 201; el fallo no provoca error 5xx.
Criticidad si falla	GRAVE

Verificaciones ejecutadas

- OK Carrier DHL: prefijo TRK-, fallback=False, entrega 2026-10-18
  `HTTP 201; tracking=TRK-47CA558F, fallback=False, entrega=2026-10-18`
- OK Carrier FAIL_CARRIER: prefijo FBK-, fallback=True, entrega 2026-10-22
  `HTTP 201; tracking=FBK-4FB30CCE, fallback=True, entrega=2026-10-22`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-05: Fallback del Circuit Breaker ante fallo del carrier



CP-06 - Búsqueda y filtrado de productos PAS

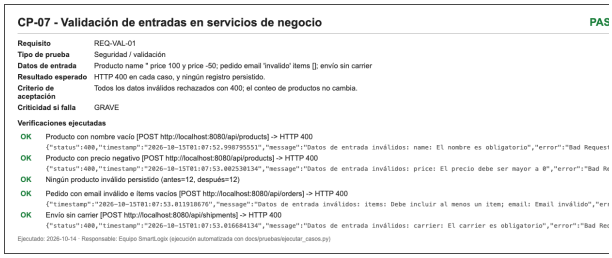
Requisito	REQ-FUNC-03
Tipo de prueba	Funcional / caja negra
Datos de entrada	Filtros: brand=Dior, fragrance=Amaderado, price-range 70000-110000, in-stock, search=SAUVAGE
Resultado esperado	Listado completo de 11 productos; cada filtro devuelve solo coincidencias.
Criterio de aceptación	Todos los elementos cumplen el criterio; búsqueda por nombre insensible a mayúsculas.
Criticidad si falla	MODERADO

Verificaciones ejecutadas

- OK Listar catálogo [GET http://localhost:8080/api/products] -> HTTP 200
  `[{"id":1,"name":"Sauvage Eau de Parfum","brand":"Dior","description":"Fragancia masculina con notas de bergamota y ámbar","price":89990.0,"stockQuant`
- OK Catálogo contiene al menos los 11 productos semilla (encontrados 12)
  `12`
- OK Filtro brand/Dior: 1 resultados, todos coinciden
  `HTTP 200, n=1`
- OK Filtro fragrance/Amaderado: 1 resultados, todos coinciden
  `HTTP 200, n=1`
- OK Filtro price-range?min=70000&max=110000: 6 resultados, todos coinciden
  `HTTP 200, n=6`
- OK Filtro in-stock: 12 resultados, todos coinciden
  `HTTP 200, n=12`
- OK Filtro search?name=SAUVAGE: 1 resultados, todos coinciden
  `HTTP 200, n=1`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-06: Búsqueda y filtrado de productos



CP-07 - Validación de entradas en servicios de negocio PAS

Requisito	REQ-VAL-01
Tipo de prueba	Seguridad / validación
Datos de entrada	Producto name " price 100 y price -50; pedido email 'invalido' items []; envío sin carrier
Resultado esperado	HTTP 400 en cada caso, y ningún registro persistido.
Criterio de aceptación	Todos los datos inválidos rechazados con 400; el conteo de productos no cambia.
Criticidad si falla	GRAVE

Verificaciones ejecutadas

- OK Producto con nombre vacío [POST http://localhost:8080/api/products] -> HTTP 400
  `{"status":400,"timestamp":"2026-10-15T01:07:52.998795551","message":"Datos de entrada inválidos: name: El nombre es obligatorio","error":"Bad Request`
- OK Producto con precio negativo [POST http://localhost:8080/api/products] -> HTTP 400
  `{"status":400,"timestamp":"2026-10-15T01:07:53.002530134","message":"Datos de entrada inválidos: price: El precio debe ser mayor a 0","error":"Bad Re`
- OK Ningún producto inválido persistido (antes=12, después=12)
- OK Pedido con email inválido e ítems vacíos [POST http://localhost:8080/api/orders] -> HTTP 400
  `{"timestamp":"2026-10-15T01:07:53.011918670","message":"Datos de entrada inválidos: items: Debe incluir al menos un ítem; email: Email inválido","er`
- OK Envío sin carrier [POST http://localhost:8080/api/shipments] -> HTTP 400
  `{"status":400,"timestamp":"2026-10-15T01:07:53.016684134","message":"Datos de entrada inválidos: carrier: El carrier es obligatorio","error":"Bad Re`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-07: Validación de entradas en servicios de negocio



CP-08 - Validación de entradas a través del gateway (BFF) PAS

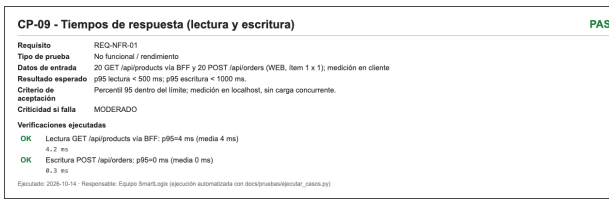
Requisito	REQ-VAL-01
Tipo de prueba	Seguridad / integración
Datos de entrada	Producto name " enviado a través del BFF con token válido
Resultado esperado	HTTP 400 propagado desde el servicio de inventario.
Criterio de aceptación	El código de error de validación llega íntegro al cliente.
Criticidad si falla	MODERADO

Verificaciones ejecutadas

- OK Producto con nombre vacío vía BFF [POST http://localhost:8080/api/products] -> HTTP 400
  `{"status":400,"timestamp":"2026-10-15T01:07:53.021470217","message":"Datos de entrada inválidos: name: El nombre es obligatorio","error":"Bad Request`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-08: Validación de entradas a través del gateway (BFF)



CP-09 - Tiempos de respuesta (lectura y escritura) PAS

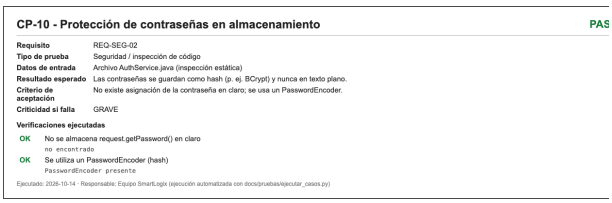
Requisito	REQ-NFR-01
Tipo de prueba	No funcional / rendimiento
Datos de entrada	20 GET /api/products vía BFF y 20 POST /api/orders (WEB, ítem 1 x 1); medición en cliente
Resultado esperado	p95 lectura < 500 ms; p95 escritura < 1000 ms.
Criterio de aceptación	Percentil 95 dentro del límite; medición en localhost, sin carga concurrente.
Criticidad si falla	MODERADO

Verificaciones ejecutadas

- OK Lectura GET /api/products vía BFF: p95=4 ms (media 4 ms)
  `4.2 ms`
- OK Escritura POST /api/orders: p95=0 ms (media 0 ms)
  `0.3 ms`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-09: Tiempos de respuesta (lectura y escritura)



CP-10 - Protección de contraseñas en almacenamiento PAS

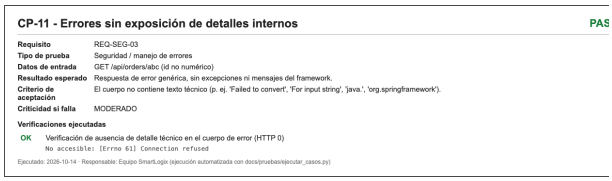
Requisito	REQ-SEG-02
Tipo de prueba	Seguridad / inspección de código
Datos de entrada	Archivo AuthService.java (inspección estática)
Resultado esperado	Las contraseñas se guardan como hash (p. ej. BCrypt) y nunca en texto plano.
Criterio de aceptación	No existe asignación de la contraseña en claro; se usa un PasswordEncoder.
Criticidad si falla	GRAVE

Verificaciones ejecutadas

- OK No se almacena request.getPassword() en claro
  `no encontrado`
- OK Se utiliza un PasswordEncoder (hash)
  `PasswordEncoder presente`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-10: Protección de contraseñas en almacenamiento



CP-11 - Errores sin exposición de detalles internos PAS

Requisito	REQ-SEG-03
Tipo de prueba	Seguridad / manejo de errores
Datos de entrada	GET /api/orders/abc (id no numérico)
Resultado esperado	Respuesta de error genérica, sin excepciones ni mensajes del framework.
Criterio de aceptación	El cuerpo no contiene texto técnico (p. ej. 'Failed to convert', 'For input string', 'java.', 'org.springframework').
Criticidad si falla	MODERADO

Verificaciones ejecutadas

- OK Verificación de ausencia de detalle técnico en el cuerpo de error (HTTP 0)
  `No accesible: [Errno 61] Connection refused`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-11: Errores sin exposición de detalles internos



CP-12 - Restricción de orígenes CORS PAS

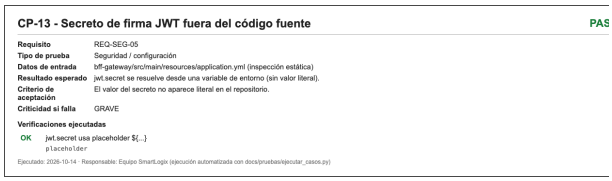
Requisito	REQ-SEG-04
Tipo de prueba	Seguridad / configuración
Datos de entrada	OPTIONS /api/products con Origin http://evil.example y http://localhost:5173
Resultado esperado	Origen no autorizado sin Access-Control-Allow-Origin; origen localhost:5173 autorizado.
Criterio de aceptación	Solo los orígenes definidos reciben la cabecera de permiso.
Criticidad si falla	MODERADO

Verificaciones ejecutadas

- OK Origen http://evil.example rechazado (HTTP 403)
  `ACAO=None`
- OK Origen http://localhost:5173 permitido (HTTP 200)
  `ACAO=http://localhost:5173`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-12: Restricción de orígenes CORS



CP-13 - Secreto de firma JWT fuera del código fuente PAS

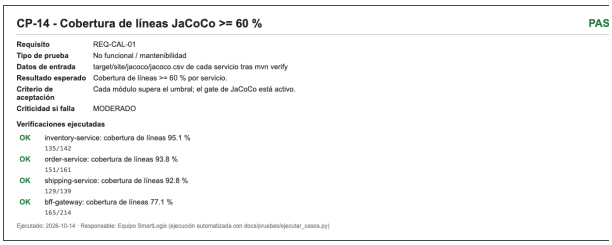
Requisito	REQ-SEG-05
Tipo de prueba	Seguridad / configuración
Datos de entrada	bff-gateway/src/main/resources/application.yml (inspección estática)
Resultado esperado	jwt.secret se resuelve desde una variable de entorno (sin valor literal).
Criterio de aceptación	El valor del secreto no aparece literal en el repositorio.
Criticidad si falla	GRAVE

Verificaciones ejecutadas

- OK jwt.secret usa placeholder ${...}
  `placeholder`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-13: Secreto de firma JWT fuera del código fuente



CP-14 - Cobertura de líneas JaCoCo >= 60 % PAS

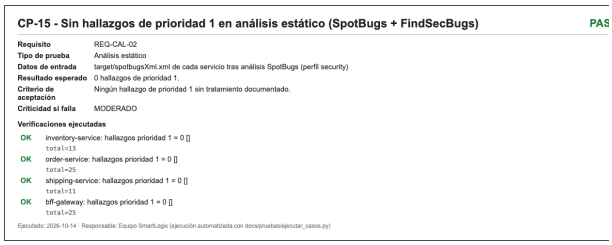
Requisito	REQ-CAL-01
Tipo de prueba	No funcional / mantenibilidad
Datos de entrada	target/site/jacoco/jacoco.csv de cada servicio tras mvn verify
Resultado esperado	Cobertura de líneas >= 60 % por servicio.
Criterio de aceptación	Cada módulo supera el umbral; el gate de JaCoCo está activo.
Criticidad si falla	MODERADO

Verificaciones ejecutadas

- OK inventory-service: cobertura de líneas 95.1 %
  `135/142`
- OK order-service: cobertura de líneas 93.8 %
  `151/161`
- OK shipping-service: cobertura de líneas 92.8 %
  `129/139`
- OK bff-gateway: cobertura de líneas 77.1 %
  `165/214`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-14: Cobertura de líneas JaCoCo >= 60 %



CP-15 - Sin hallazgos de prioridad 1 en análisis estático (SpotBugs + FindSecBugs) PAS

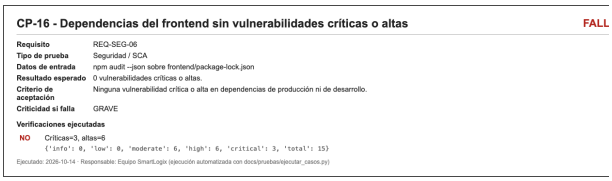
Requisito	REQ-CAL-02
Tipo de prueba	Análisis estático
Datos de entrada	target/spotbugsXml.xml de cada servicio tras análisis SpotBugs (perfil security)
Resultado esperado	0 hallazgos de prioridad 1.
Criterio de aceptación	Ningún hallazgo de prioridad 1 sin tratamiento documentado.
Criticidad si falla	MODERADO

Verificaciones ejecutadas

- OK inventory-service: hallazgos prioridad 1 = 0 []
  `total=13`
- OK order-service: hallazgos prioridad 1 = 0 []
  `total=25`
- OK shipping-service: hallazgos prioridad 1 = 0 []
  `total=11`
- OK bff-gateway: hallazgos prioridad 1 = 0 []
  `total=25`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-15: Sin hallazgos de prioridad 1 en análisis estático (SpotBugs + FindSecB



CP-16 - Dependencias del frontend sin vulnerabilidades críticas o altas FALL

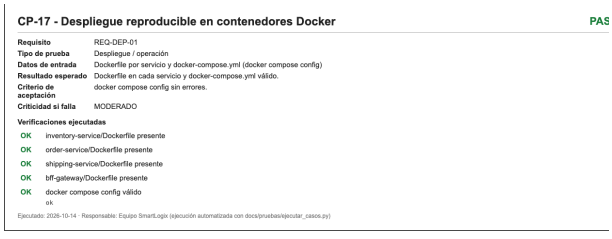
Requisito	REQ-SEG-06
Tipo de prueba	Seguridad / SCA
Datos de entrada	npm audit --json sobre frontend/package-lock.json
Resultado esperado	0 vulnerabilidades críticas o altas.
Criterio de aceptación	Ninguna vulnerabilidad crítica o alta en dependencias de producción ni de desarrollo.
Criticidad si falla	GRAVE

Verificaciones ejecutadas

- NO Críticas=3, altas=6
  `{'info': 0, 'low': 0, 'moderate': 6, 'high': 6, 'critical': 3, 'total': 15}`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-16: Dependencias del frontend sin vulnerabilidades críticas o altas



CP-17 - Despliegue reproducible en contenedores Docker PAS

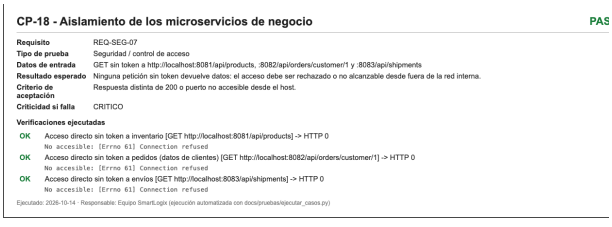
Requisito	REQ-DEP-01
Tipo de prueba	Despliegue / operación
Datos de entrada	Dockerfile por servicio y docker-compose.yml (docker compose config)
Resultado esperado	Dockerfile en cada servicio y docker-compose.yml válido.
Criterio de aceptación	docker compose config sin errores.
Criticidad si falla	MODERADO

Verificaciones ejecutadas

- OK inventory-service/Dockerfile presente
- OK order-service/Dockerfile presente
- OK shipping-service/Dockerfile presente
- OK bff-gateway/Dockerfile presente
- OK docker compose config válido
  `ok`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

CP-17: Despliegue reproducible en contenedores Docker



CP-18 - Aislamiento de los microservicios de negocio PAS

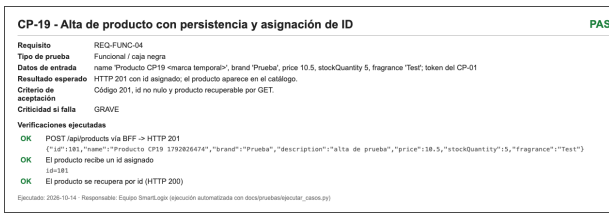
Requisito	REQ-SEG-07
Tipo de prueba	Seguridad / control de acceso
Datos de entrada	GET sin token a http://localhost:8081/api/products, :8082/api/orders/customer/1 y :8083/api/shipments
Resultado esperado	Ninguna petición sin token devuelve datos: el acceso debe ser rechazado o no alcanzable desde fuera de la red interna.
Criterio de aceptación	Respuesta distinta de 200 o puerto no accesible desde el host.
Criticidad si falla	CRITICO

Verificaciones ejecutadas

- OK Acceso directo sin token a inventario [GET http://localhost:8081/api/products] -> HTTP 0
  `No accesible: [Errno 61] Connection refused`
- OK Acceso directo sin token a pedidos (datos de clientes) [GET http://localhost:8082/api/orders/customer/1] -> HTTP 0
  `No accesible: [Errno 61] Connection refused`
- OK Acceso directo sin token a envíos [GET http://localhost:8083/api/shipments] -> HTTP 0
  `No accesible: [Errno 61] Connection refused`

Ejecutado: 2026-10-14 · Responsable: Equipo SmartLogix (ejecución automatizada con docs/pruebas/ejecutar_casos.py)

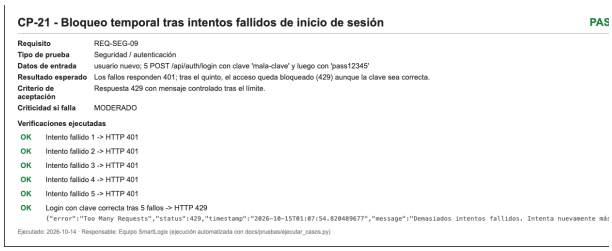
CP-18: Aislamiento de los microservicios de negocio



CP-19: Alta de producto con persistencia y asignación de ID



CP-20: Microservicio rechaza peticiones sin token desde la red interna



CP-21: Bloqueo temporal tras intentos fallidos de inicio de sesión



CP-22: Actualización de estado de pedido a través del BFF



CP-23: Parámetro obligatorio ausente devuelve 400

Anexo B. Reproducibilidad

cp .env.example .env # definir JWT_SECRET (openssl rand -base64 64)

docker compose up -d --build

./docs/pruebas/esperar_stack.sh 240

python3 docs/pruebas/ejecutar_casos.py <etiqueta>

cd e2e && npm ci && npx playwright test

./docs/seguridad/ejecutar_zap.sh <etiqueta>

cd <servicio> && mvn -B clean verify

cd <servicio> && mvn -Psecurity com.github.spotbugs:spotbugs-maven-plugin:4.8.6.6:spotbugs

cd frontend && npm ci && npx vitest run --coverage && npm audit --omit=dev

python3 docs/informe/generar_informe_encargo3.py

Anexo C. Mapa de evidencias

Archivo	Contenido
docs/evidencias/main-encargo2-stack/resultados.json	Batería de 23 casos sobre la versión de referencia
docs/evidencias/v2-final/resultados.json	Batería de 23 casos sobre la versión final
docs/evidencias/encargo3/e2e-linea-base-main.txt	Salida de E2E y API en la versión de referencia
docs/evidencias/encargo3/playwright-resultados-v2.json	Resultados de E2E y API en la versión final

docs/seguridad/zap/main y v2-final	Reportes ZAP (HTML y JSON) de ambas versiones
docs/evidencias/encargo3/sonar-*.json	Métricas y quality gate de SonarQube
docs/evidencias/encargo3/npm-audit-*.json	Auditoría de dependencias (producción y completa)
docs/evidencias/encargo3/spotbugs-*.xml y jacoco-*.csv	Hallazgos de análisis estático y cobertura por módulo
docs/evidencias/encargo3/A01 a A03, S01 a S03, Z01, Z02, E01, D01	Capturas de GitHub Actions, SonarQube, ZAP, Playwright y del pipeline
docs/evidencias/capturas/UI-01 a UI-06	Capturas del flujo en el navegador
e2e/tests, .github/workflows/ci.yml, docs/pruebas	Código de las pruebas y del pipeline

Anexo D. Responsables de la ejecución

Asignación de responsabilidades dentro del grupo. Se define como propuesta del equipo y debe confirmarse con cada integrante antes de la defensa, porque la defensa es individual y cada uno debe poder explicar el conjunto.

Integrante	Responsabilidad en el Encargo 3
Nicolás Castillo	Stack Docker, mitigaciones del backend, análisis SpotBugs y pipeline de CI/CD.
Benjamín Catalán	Análisis dinámico (ZAP), pruebas E2E y de API, y capturas de la interfaz.
Benjamín Quiroz	Análisis estático (SonarQube y ESLint), auditoría de dependencias y revisión normativa.